Unit-5: Image Filtering Operations & Computer Vision using OpenCV

Overview

This unit explores the practical side of computer vision using OpenCV, with a focus on applying image filtering operations and object detection techniques such as Haar cascades. Students will learn how to manipulate images and detect features like edges, motion blur, and facial structures in real-world images and videos.

Image Filtering

What is Digital Image?

- Digital images are made of picture elements called pixels.
- Typically, pixels are organized in an ordered rectangular array.
- The size of an image is determined by the dimensions of this pixel array.
- The image width is the number of columns, and the image height is the number of rows in the array.
- Thus, the pixel array is a matrix of M rows x N Columns.

What is meant by Convolutional?

Word 'Convolutional' comes from the mathematical operation called convolution, which is the core idea behind how CNNs (Convolutional Neural Networks) understand images.

In Simple Terms, Convolutional means scanning small patterns (filters) over an image to detect features like ***edges, corners, shapes, etc***.

- Convolution is a simple mathematical operation.
- It is a common image processing operator.
- Convolution provides a way of multiplying together two arrays of numbers, that are generally of different sizes but of the same dimensionality to produce a third array of numbers of the same dimensionality.
- It uses a matrix called ***kernel*** to change the pixel value by applying a mathematical operator.
- Generally, kernel size must be smaller than image size.
- The process of applying a kernel on the image is known as ***Image Filtering*** and the output image is known as ***Filtered Image***.
- It's what allows CNNs to "see" like a human would see.

What is a Convolution/2D Convolution Operation?

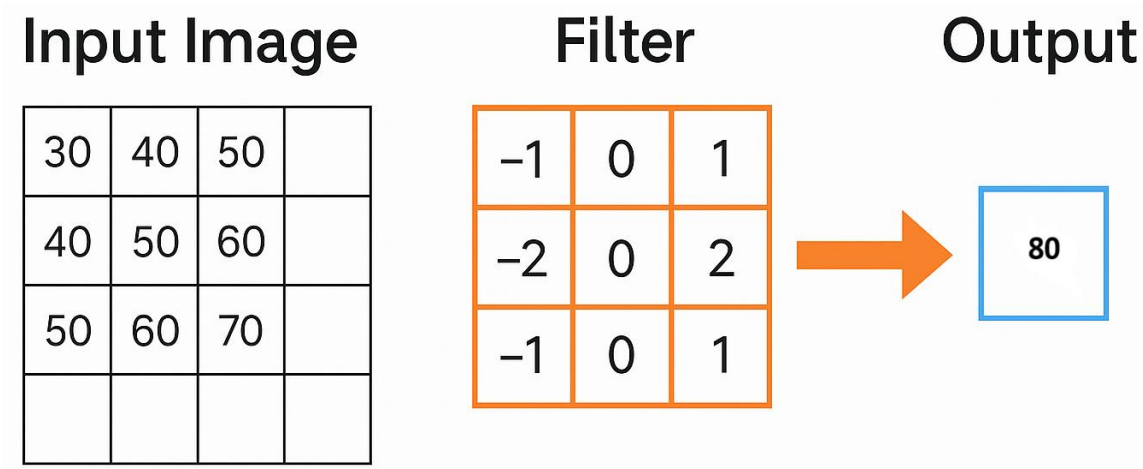
Convolution = *Matrix Multiplication of a Filter with a Part of the Image.*

Example:
Let's take an image patch of (3x3):

[30, 40, 50]
[40, 50, 60]
[50, 60, 70]

Kernel Filter

[-1, 0, 1]
[-2, 0, 2]
[-1, 0, 1]



Calculation

Step-1

(30×-1) + (40×0) + (50×1) +
(40×-2) + (50×0) + (60×2) +
(50×-1) + (60×0) + (70×1)

Step-2

= -30 + 0 + 50
+ -80 + 0 + 120
+ -50 + 0 + 70
= (20) + (40) + (20) = **80**

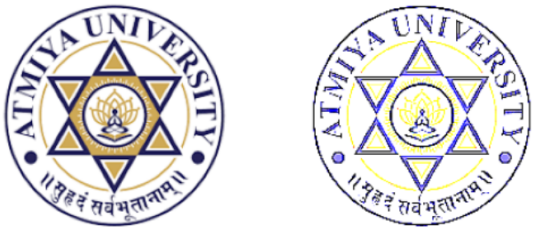
So, to sum up...

Term	Meaning
Convolution	Sliding a small filter over an image and performing element-wise math
Kernel/Filter	Small matrix used for feature detection
Image Filtering	The process of applying a kernel on the image
Filtered image	Output image after convolution, showing detected features

Applying Kernel to an image (2d Filtering):

```python
import cv2
import matplotlib.pyplot as plt
import numpy as np
img=cv2.imread('au.png')
img=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
# k1 = np.array([[0,0,0],[0,1,0],[0,0,0]])
k1 = np.array([[-1,-1,-1],[-1,12,-1],[-1,-1,-1]])
#cv2.filter2D() is an OpenCV function used to apply a custom filter (kernel) to an image using 2D convolution.
#syntax:cv2.filter2D(src, ddepth, kernel)
k1output = cv2.filter2D(img,-1,k1)
plt.subplot(1,2,1),plt.imshow(img),plt.axis('off')
plt.subplot(1,2,2),plt.imshow(k1output),plt.axis('off')
```

```
(<Axes: >,
 <matplotlib.image.AxesImage at 0x25aa85d6f20>,
 (np.float64(-0.5), np.float64(203.5), np.float64(191.5), np.float64(-0.5)))
```



Blurring an image

- Blur in the image processing refers to the reduction of sharpness or clarity in an image.
- It's a technique that involves spreading pixels in an image to reduce details and noise.
- This can be useful for various image processing tasks, such as smoothing an image, reducing the visibility of imperfections and defects, and decreasing noise in an image.
- Blur can be applied in various ways, such as through specific filters that calculate the average of neighboring pixels or through more advanced techniques like convolution with Gaussian kernels.
- The result is an image that appears smoother or less sharp than the original.

```python
#Bluring an image
import cv2
import matplotlib.pyplot as plt
import numpy as np
img=cv2.imread('au.png')
img=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
#Image filtering
k1=np.ones((4,4),np.float32)/16
img1=cv2.filter2D(img,-1,k1)
plt.subplot(1,2,1),plt.imshow(img),plt.axis('off'),plt.title('Original')
plt.subplot(1,2,2),plt.imshow(img1),plt.axis('off'),plt.title('Blured')
```

```
(<Axes: title={'center': 'Blured'}>,
 <matplotlib.image.AxesImage at 0x25aa8c47b80>,
 (np.float64(-0.5), np.float64(203.5), np.float64(191.5), np.float64(-0.5)),
 Text(0.5, 1.0, 'Blured'))
```

Types of Blurs:

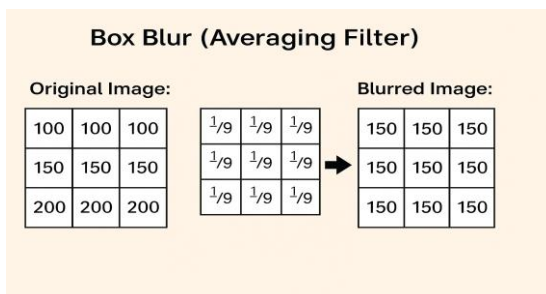
1. Box Blur
2. Gaussian Blur

3. Median Blur
4. Motion Blur

1. Box Blur

- It calculates the average of pixels in a rectangular area around each pixel, resulting in uniform smoothing.
- Box blur is a simpler approach.
- Box blur simply averages the pixel values within a rectangular or square area around the central pixel.
- For each pixel in the image, a "box" of defined size is created around it, and all pixel values within this box are summed and divided by the total number of pixels in the box.
- This resulting value is then assigned to the central pixel in the new image, thus creating the smoothing effect.
- While box blur is easier to calculate and generally faster than Gaussian blur, it can result in visible artifacts, especially in images with fine details or smooth transitions.
- For this reason, Gaussian blur is often preferred when visual quality is crucial, while box blur can be used in situations where computational efficiency is more important.

Example:



Sum of all values:

$$100 + 100 + 100 + 150 + 150 + 150 + 200 + 200 + 200 = 1350$$

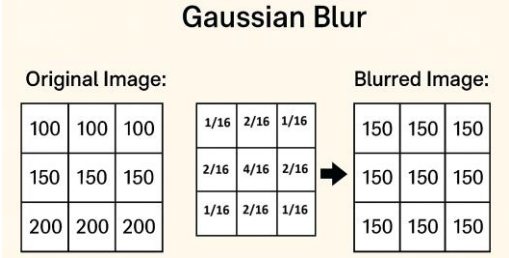
Average =

$$\frac{1350}{9} = 150$$

2. Gaussian Blur

- It utilizes a Gaussian distribution function to smooth the image, giving more weight to pixels near the center of the filter.
- The name comes from the Gaussian distribution technique, also known as a "bell-shaped curve," which is used to calculate the blur intensity in different parts of the image.
- In simple terms, Gaussian blur works by calculating the weighted average of neighboring pixel values in an image.
- For each pixel in the image, a square or circular "window" is defined around it, and the pixel values within this window are weighted according to their proximity to the central pixel.
- The farther a pixel is from the central pixel, the less weight it has in the average calculation.
- The result is a smoothed image where fine details are attenuated, and color transitions are smoother.

Example:



Let's compute weighted sum:

$$= 100 \times 1 + 100 \times 2 + 100 \times 1 + 150 \times 2 + 150 \times 4 + 150 \times 2 + 200 \times 1 + 200 \times 2 + 200 \times 1$$

$$= 100 \times (1 + 2 + 1) + 150 \times (2 + 4 + 2) + 200 \times (1 + 2 + 1)$$
$$= 100 \times 4 + 150 \times 8 + 200 \times 4$$
$$= 400 + 1200 + 800 = 2400$$

Final Value:

$$\frac{2400}{16} = 150$$

Feature	**Box Blur**	**Gaussian Blur**
Sharpness loss	Harsher	Softer, more natural
Edge preservation	Poor	Better (edges are smoother)
Blur shape	Uniform square	Smooth, circular fall-off

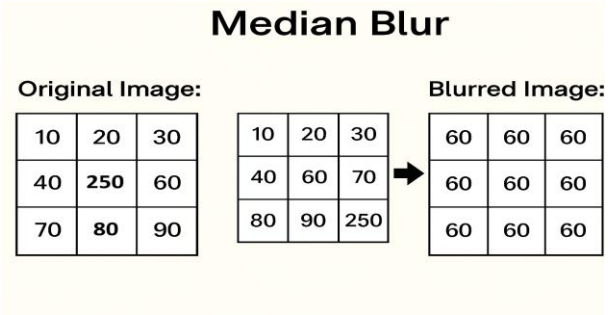
<u>Real Difference Is in the Shape of the Blur</u>

- Box Blur = Like pulling a square brush across the image
- Gaussian Blur = Like feathering with a soft circular brush

<u>3. Median Blur</u>

- The Median blur operation is like the other averaging methods.
- Here, the central element of the image is replaced by the median of all the pixels in the kernel area.
- This operation processes the edges while removing the noise.

Example:

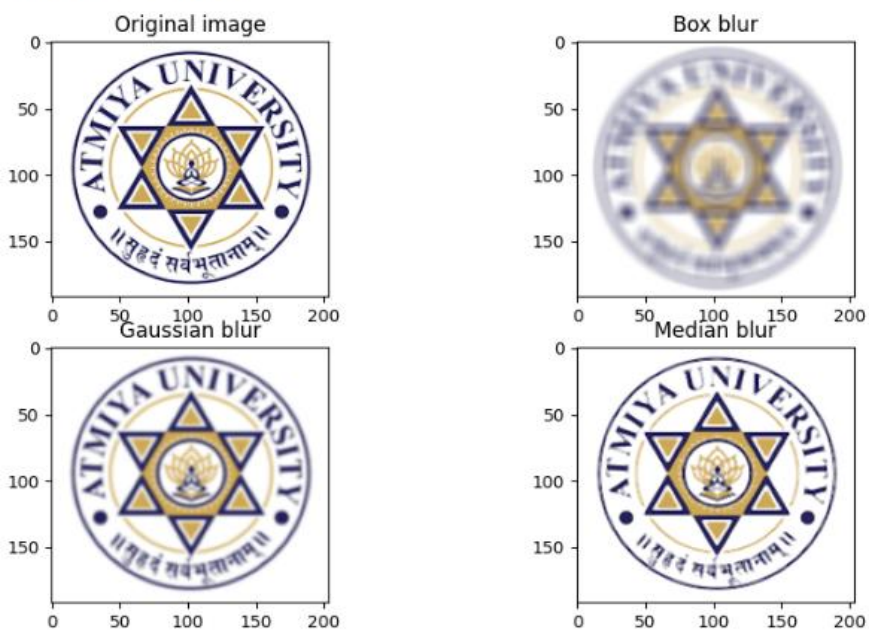


```
Original 3x3 Patch:
[[ 10  20  30]
 [ 40 250  60]
 [ 70  80  90]]

Flattened & Sorted:
[ 10  20  30  40  60  70  80  90 250]

Median Value (replaces center): 60
```

Type	Method Used	Handles Noise Well?	Preserves Edges?
Box Blur	Equal average	No	No
Gaussian Blur	Weighted average	Moderate	Yes
Median Blur	Median value	Excellent	Very Good

```python
#Applpying different types of blur effect...
import cv2
import matplotlib.pyplot as plt
import numpy as np
img=cv2.imread('au.png')
img=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
#Box blur
box=cv2.blur(img,(10,10)) #it can be (3,3), (5,5), (10,10) or (20,20)
#Gaussian blur
gaus=cv2.GaussianBlur(img,(5,5),0) #it can be any odd nos. like (3,3),(5,5),(7,7) like wise.
                                   #0 means, automatically calculate the blur effect. Otherwise we can pass any number.
#Median blur
medi=cv2.medianBlur(img,3)#It can be any odd number greater than 1
plt.figure(figsize=(10,6))
plt.subplot(2,2,1),plt.imshow(img),plt.title('Original image')
plt.subplot(2,2,2),plt.imshow(box),plt.title('Box blur')
plt.subplot(2,2,3),plt.imshow(gaus),plt.title('Gaussian blur')
plt.subplot(2,2,4),plt.imshow(medi),plt.title('Median blur')
```

```
(<Axes: title={'center': 'Median blur'}>,
 <matplotlib.image.AxesImage at 0x25aac77aec0>,
 Text(0.5, 1.0, 'Median blur'))
```



4. Motion Blur

- In this filter, the output image looks like it is captured while moving in a particular direction.
- Ex. Image captured from moving vehicle.
- The kernel is created in such a way that the pixel value is average of every pixel in particular direction.
- This filter is also known as Directional Low Pass Filter.
- To blur image in horizontal direction below kernel can be used:

$$K = \frac{1}{3}\begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

- To blur image in vertical direction below kernel can be used:

$$K = \frac{1}{3}\begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

```python
#Motion Blur
import cv2
import matplotlib.pyplot as plt
import numpy as np

img=cv2.imread('tree1.jpg')
img=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)

size=30 # Sets the size of the kernel to 30×30

#Horizontal blur
k1=np.zeros((size,size)) #Creates a 30×30 matrix filled with zeros
k1[int(size/2),:]=np.ones(size) # changing only the middle row (row number 15 out of 0-29) to all 1s
k1=k1/size # Each 1 becomes 1/30 ≈ 0.033

#Vertical blur
k2=np.zeros((size,size)) #Creates a 30×30 matrix filled with zeros
k2[:,int(size/2)]=np.ones(size) # changing only the middle column (column number 15 out of 0-29) to all 1s
k2=k2/size # Each 1 becomes 1/30 ≈ 0.033

#Applying the Kernel
horizontal_img=cv2.filter2D(img,-1,k1)
vertical_img=cv2.filter2D(img,-1,k2)

#Ploting the figure
plt.figure(figsize=(10,6))
plt.subplot(1,3,1),plt.imshow(img),plt.title('Original image')
plt.subplot(1,3,2),plt.imshow(horizontal_img),plt.title('Horizontal blur')
plt.subplot(1,3,3),plt.imshow(vertical_img),plt.title('Vertical blur')
```

```
(<Axes: title={'center': 'Vertical blur'}>,
 <matplotlib.image.AxesImage at 0x156d6ea3c40>,
 Text(0.5, 1.0, 'Vertical blur'))
```

Edge Detection

Edge: In digital images, edges correspond to abrupt changes in pixel brightness or color.

Edge detection is a key technique in image processing and computer vision.
It identifies sharp changes in pixel intensity, which typically correspond to:

- Boundaries of objects
- Lines, corners, or curves
- Transitions between colors or textures

In simple terms, it finds where things start and stop in an image.

This filter is used to detect sharp edges in the image.
It is also known as High Pass Filter Operation.
It detects the edges by retaining the edges from the image and discarding the remaining portion.

Why Is Edge Detection Important?
Edge detection is the process of identifying significant local changes in intensity (pixel values) within an image. These changes often represent:

- Object boundaries
- Shape outlines
- Surface discontinuities

Edge detection is used for:
- Object detection and recognition
- Image segmentation
- Facial recognition
- Robot navigation
- Medical imaging

How Edge Detection works?
Edges occur in areas with high contrast — a rapid change in brightness.
To detect them:
1. We calculate the ***gradient*** (rate of change) of pixel values.
2. Areas with high ***gradient*** are marked as edges.

Gradient: The gradient of an image represents the rate and direction of change in pixel intensity.

The real meaning of Gradient in Images:

Area Type	Gradient Value	What It Means
Flat region	Low/zero	No change in brightness
Edge	High	Sharp change in brightness
Texture/Corner	Very high	Rapid changes in multiple directions

Below is the sample image by which we can understand what gradient is:

Let's understand it with number:
[10, 10, **10, 200**, 200, 200]
Observing the number, we can see that there is a sharp jump from 10 to 200 → the gradient is very high → this is an edge.

There are three main types of Edge Detection filters:
1. Sobel (horizontal/Vertical)
2. Laplacian
3. Canny

1. **Sobel (Horizontal/Vertical)**

- Sobel edge detection is one of the simplest and oldest filters for edge detection.
- It uses two kernels/filters, one for horizontal and one for vertical edges, that are convolved with the image.
- The kernels are designed to measure the gradient or the rate of the change of pixel intensity.
- The sobel filter then combines the horizontal and vertical gradient to obtain the edge magnitude and direction.
- It's easy to implement, fast to compute and robust to noise.
- The disadvantages are that it is sensitive to diagonal edges, produces thick edges and does not account for edge continuity or smoothness.

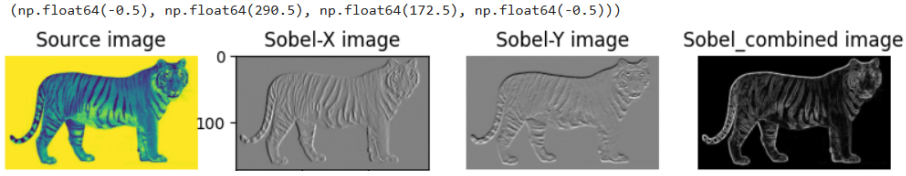
Syntax:

cv2.Sobel(src, ddepth, dx, dy, ksize)

Parameter	Description
src	Source image (grayscale)
ddepth	Desired depth of the output image (e.g., cv2.CV_64F)
dx	Order of derivative in x-direction (1 for Sobel X)
dy	Order of derivative in y-direction (1 for Sobel Y)
ksize	Kernel size: must be 1, 3, 5, or 7 (3 is default)

```
#Edge Detection
import cv2
import matplotlib.pyplot as plt
img=cv2.imread('tiger.jpg')
img=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY) #Edge detection works on intensity values, not on color. So, COLOR_BGR2GRAY is used.

#Sobel(image,ddepth,x,y,kernel_size)
sobelx=cv2.Sobel(img,cv2.CV_64F,1,0,10)
sobely=cv2.Sobel(img,cv2.CV_64F,0,1,10)
sobel_combined = cv2.magnitude(sobelx, sobely) #The magnitude combines strong edges, regardless of horizontal or vertical.

#Ploting the image
plt.figure(figsize=(10,6))
plt.subplot(1,4,1),plt.imshow(img),plt.title('Source image'),plt.axis('off')
plt.subplot(1,4,2),plt.imshow(sobelx,cmap='gray'),plt.title('Sobel-X image') #cmap='gray', tells matplotlib to render the image in grayscale
plt.subplot(1,4,3),plt.imshow(sobely,cmap='gray'),plt.title('Sobel-Y image'),plt.axis('off')
plt.subplot(1,4,4),plt.imshow(sobel_combined,cmap='gray'),plt.title('Sobel_combined image'),plt.axis('off')
```

```
(<Axes: title={'center': 'Sobel_combined image'}>,
 <matplotlib.image.AxesImage at 0x20ba22e3c40>,
 Text(0.5, 1.0, 'Sobel_combined image'),
 (np.float64(-0.5), np.float64(290.5), np.float64(172.5), np.float64(-0.5)))
```



When to use Sobel:

Use Case	Purpose
Preprocessing	Extract object boundaries
OCR/Face Detection	Find structural edges
Autonomous Vehicles	Lane detection, road edges

To sum up...

Term	Meaning
Sobel X	Detects vertical edges (left/right changes)
Sobel Y	Detects horizontal edges (up/down changes)
Combined	Detects edges in all directions
Used in	OCR, object detection, shape recognition

2. Laplacian

- This filter uses a single kernel, usually a 3x3 or a 5x5 one, that is convolved with the image.
- It measures the curvature or the rate of change of the gradient, which indicates the presence of an edge.
- The kernel is designed to have a zero sum, meaning that it has positive values in the center and negative values in the surrounding pixels.
- This way, it enhances the edges and suppresses the flat regions.
- It detects edges in all directions equally, and that it produces thin and sharp edges.
- It gives good results when image is blurred first using Gaussian blur and then the edge is detected using the Laplacian.

Syntax:

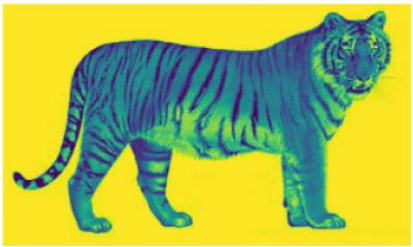
cv2.Laplacian(src, ddepth, ksize=1)

Parameter	Description
src	Input image (grayscale recommended)
ddepth	Output image data type (e.g., cv2.CV_64F)
ksize	Optional kernel size (1, 3, 5...)

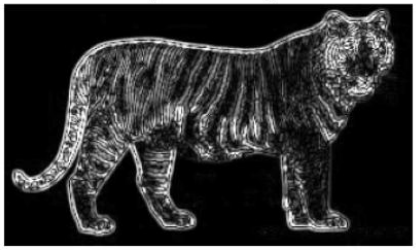
```python
#Edge Detection
import cv2
import matplotlib.pyplot as plt
img=cv2.imread('tiger.jpg')
img=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY) #Edge detection works on intensity values, not on color. So, COLOR_BGR2GRAY is used

#Gaussian Blur
Gaussian = cv2.GaussianBlur(img, (3,3), 0) #Gives good results, if the image is blurred using Gaussian blur

#Laplacian(src, ddepth, ksize=1)
laplacian=cv2.Laplacian(Gaussian,cv2.CV_64F, ksize=3)
laplacian = cv2.convertScaleAbs(laplacian)

#Ploting the image
plt.figure(figsize=(10,6))
plt.subplot(1,2,1),plt.imshow(img),plt.title('Source image'),plt.axis('off')
plt.subplot(1,2,2),plt.imshow(laplacian,cmap='gray'),plt.title('Laplacian image'),plt.axis('off')
```

```
(<Axes: title={'center': 'Laplacian image'}>,
 <matplotlib.image.AxesImage at 0x20ba2b92200>,
 Text(0.5, 1.0, 'Laplacian image'),
 (np.float64(-0.5), np.float64(290.5), np.float64(172.5), np.float64(-0.5)))
```

Source image



Laplacian image



When to use Laplacian?

Use Case	Purpose
OCR/Document Scanning	Detect characters and boundaries
Medical Imaging	Highlight tumours or blood vessels
Industrial Vision	Detect surface defects

3. Canny

- It is a more sophisticated and widely used filter for edge detection.
- It's like a "smart edge detector" that finds only true, thin, and connected edges.
- This filter consists of four main steps:
  - smoothing
  - gradient calculation
  - non-maximum suppression
  - hysteresis thresholding.
- Smoothing is done by applying a Gaussian filter to the image to reduce noise.
- Gradient calculation is like the Sobel filter but using a larger kernel size.
- Non-maximum suppression (NMS) is used to eliminate duplicate detections and select the most relevant detected objects.
- This helps reduce false positives and the computational complexity of a detection algorithm.

- Hysteresis thresholding is a technique that uses two threshold values, a lower and a higher one, to decide whether a pixel is an edge or not.
- Pixels above the higher threshold are marked as strong edges.
- Pixels below the lower threshold are discarded, and pixels in between are marked as weak edges. Weak edges are only kept if they are connected to strong edges, otherwise they are removed.
- This filter produces thin, smooth, and accurate edges.

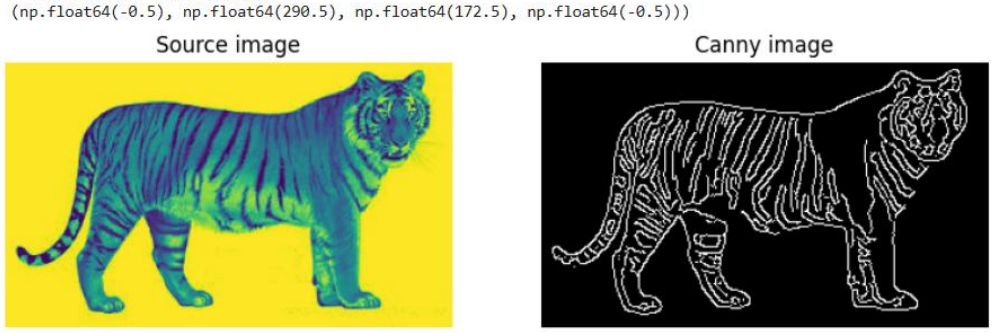
Syntax:

cv2.Canny(image, threshold1, threshold2)

Parameter	Description
threshold1	Lower threshold (weak edges)
threshold2	Upper threshold (strong edges)

```
#Edge Detection
import cv2
import matplotlib.pyplot as plt
img=cv2.imread('tiger.jpg')
img=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY) #Edge detection works on intensity values, not on color. So, COLOR_BGR2GRAY is used.

#Gaussian Blur
Gaussian = cv2.GaussianBlur(img, (5,5), 2) #Gives good results, if the image is blurred using Gaussian blur

#Canny(image, threshold1, threshold2)
canny=cv2.Canny(Gaussian,threshold1=50,threshold2=150)

#Ploting the image
plt.figure(figsize=(10,6))
plt.subplot(1,2,1),plt.imshow(img),plt.title('Source image'),plt.axis('off')
plt.subplot(1,2,2),plt.imshow(canny,cmap='gray'),plt.title('Canny image'),plt.axis('off')
```

```
(<Axes: title={'center': 'Canny image'}>,
 <matplotlib.image.AxesImage at 0x20ba2fcc610>,
 Text(0.5, 1.0, 'Canny image'),
 (np.float64(-0.5), np.float64(290.5), np.float64(172.5), np.float64(-0.5)))
```



When to use Canny?

Use Case	Purpose
Medical Imaging	Detect blood vessels or tumours

Use Case	Purpose
Robotics	Object boundary detection
OCR	Clean characters from document scans
Surveillance	Detect movement edges (intruders)

Comparison summary of Edge Detection filters

Method	Type	Pros	Cons
Sobel	First-order	Simple, detects direction	Sensitive to noise
Laplacian	Second-order	Multi-directional	Double edges, noisy
Canny	Multi-stage	Best results, accurate	Slower, more complex

First order derivative and Second order derivative

- First-order derivative → detects edges (sudden changes in brightness).
- Second-order derivative → detects where those changes in edges happen (e.g., corners, double edges)

```
Pixel intensity:
     ^
     |
 ^   |       *
 |   |     *   *
 |   |    *     *
 |   |   *       *
 +----+----------------------> pixel position
```

First Derivative (like Sobel)

- The first derivative tells you how steep the slope is.
- When you're going up the hill, the slope is positive.
- At the top, it's zero (flat).
- When you're going down, the slope is negative.
- This is what Sobel detects, where brightness is increasing or decreasing sharply.

Second Derivative (like Laplacian)

- The second derivative tells you how the slope is changing.
- It captures sharp peaks, where the rate of brightness changes shifts suddenly.
- This is like the curvature of the hill.

- This is what Laplacian detects, more sensitive to fine detail, changes in changes.

Original Image

Sobel-First Order

Laplacian-Second Order

Comparing the First order and Second order

Property	First-Order (e.g., Sobel)	Second-Order (e.g., Laplacian)
Measures	Change in pixel values	Change in rate of change
Detects	Directional edges	All-direction edges
Derivative Used	1st derivative	2nd derivative
Direction Sensitive?	Yes (X/Y)	No
Output	Gradient image	Zero-crossing/double-edge map
Noise Sensitivity	Moderate	High (usually used with blur)

Image Sharpening

Image sharpening is an image enhancement technique used to highlight fine details and edges within an image. It makes an image appear clearer and more defined by enhancing the transitions between different intensity levels (i.e., the edges).

How Does Image Sharpening Work?

- It is based on the idea of increasing the contrast of adjacent pixels.
- Sharpening can be implemented by using a high-pass filter, which preserves or enhances high-frequency components (edges and details).
- This is typically done through convolution with a sharpening kernel (filter).

One of the most used sharpening filters is:

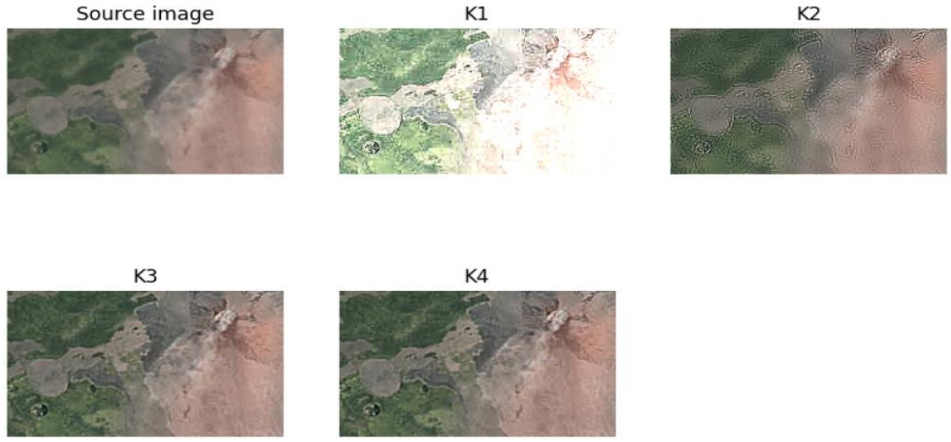
$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Explanation:
The centre pixel is multiplied by +5 (emphasizing the main pixel).
The immediate neighbours (top, bottom, left, right) are multiplied by -1 (subtracted).
This enhances intensity differences and sharpens edges.

Technique	Effect	Goal
Blurring	Smoothens the image	Removes noise/details
Sharpening	Increases contrast and edge clarity	Highlights feature

```
#Image Sharpening...
import cv2
import matplotlib.pyplot as plt
import numpy as np

img=cv2.imread('images.jpg')
img=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)

k1 = np.array([[-1,-1,-1],[-1,10,-1],[-1,-1,-1]])
k2 = np.array([[1,1,1],[1,-7,1],[1,1,1]])
k3 = np.array([[-1,-1,-1,-1,-1],[-1,2,2,2,-1],[-1,2,8,2,-1],[-1,2,2,2,-1],[-1,-1,-1,-1,-1]])/8
k4 = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])
img1 = cv2.filter2D(img,-1,k1)
img2 = cv2.filter2D(img,-1,k2)
img3 = cv2.filter2D(img,-1,k3)
img4 = cv2.filter2D(img,-1,k4)

plt.figure(figsize=(10,6))
plt.subplot(2,3,1),plt.imshow(img),plt.title('Source image'),plt.axis('off')
plt.subplot(2,3,2),plt.imshow(img1),plt.title('K1'),plt.axis('off')
plt.subplot(2,3,3),plt.imshow(img2),plt.title('K2'),plt.axis('off')
plt.subplot(2,3,4),plt.imshow(img3),plt.title('K3'),plt.axis('off')
plt.subplot(2,3,5),plt.imshow(img3),plt.title('K4'),plt.axis('off')
```

```
(<Axes: title={'center': 'K4'}>,
 <matplotlib.image.AxesImage at 0x14a990cb010>,
 Text(0.5, 1.0, 'K4'),
 (np.float64(-0.5), np.float64(285.5), np.float64(175.5), np.float64(-0.5)))
```



Applications of Image Sharpening

- Medical imaging (e.g., enhancing X-rays, MRIs)
- Satellite and aerial imaging
- Document image enhancement
- Object detection and edge-based segmentation
- Photo enhancement and restoration

Gaussian Kernel for more natural image enhancement

Gaussian kernels apply a weighted blur that gives more importance to pixels near the center of the kernel and less to those further away. They are based on the Gaussian distribution (bell curve).

Syntax:

cv2.addWeighted(src1, alpha, src2, beta, gamma)

Conceptual application:

Unsharp Image = 1.5 x Original Image − 0.5 x Blurred Image

Image Embossing

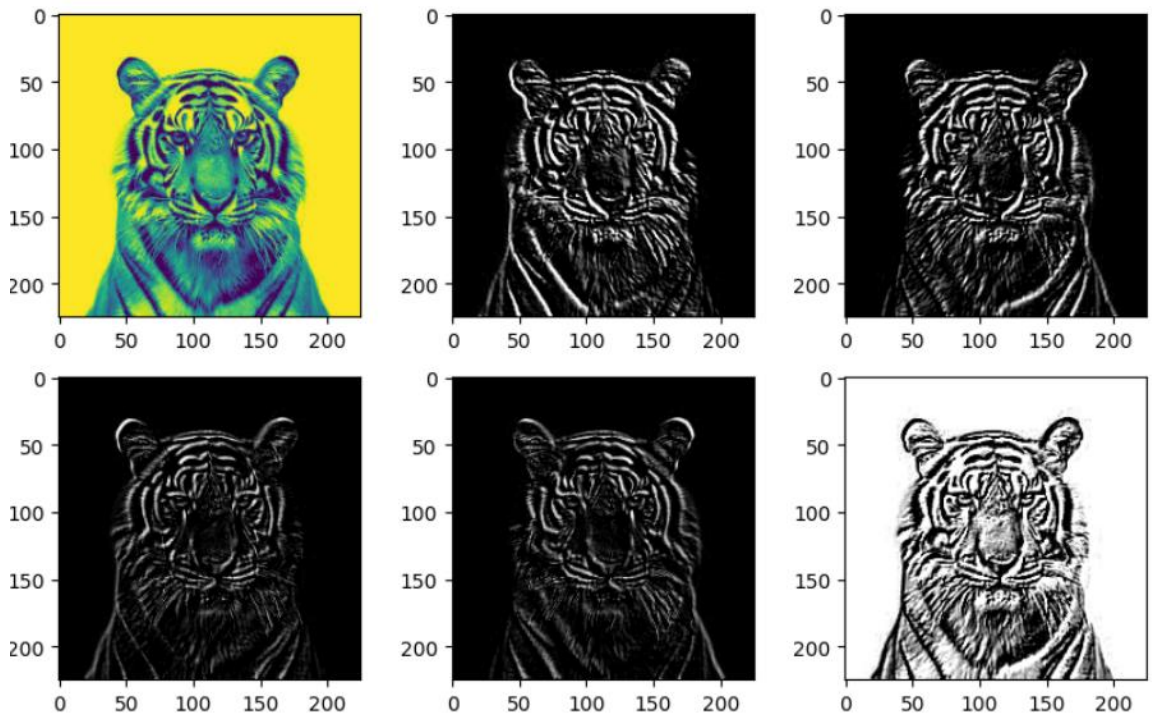
Image embossing is an image processing technique that creates a 3D shadow effect, making it look like the image is raised or engraved. It highlights edges with directional lighting, usually making the image appear engraved or popping out from the background.

Purpose of Embossing in Image Processing:

The purpose of an image embossing is not for beautifying an image, it's for extracting structure and texture.
- It reveals roughness in the surface
- Show irregularities that are invisible to the eye

- Highlight texture flows (like in fingerprint)
- To generate metallic like textures
- Emphasize outlines of shapes

```
#Image embossing...
import cv2
import matplotlib.pyplot as plt
import numpy as np

img = cv2.imread('tigerface.jpg', cv2.IMREAD_GRAYSCALE)

k1 = np.array([[0,-1,-1],[1,0,-1],[1,1,0]])
k2 = np.array([[-1,-1,0],[-1,0,1],[0,1,1]])
k3 = np.array([[1,0,0],[0,0,0],[0,0,-1]])
k4 = np.array([[0,0,1],[0,0,0],[-1,0,0]])
k5 = np.array([[-2, -1, 0],[-1,  1, 1],[ 0,  1, 2]])

img1 = cv2.filter2D(img,-1,k1)
img2 = cv2.filter2D(img,-1,k2)
img3 = cv2.filter2D(img,-1,k3)
img4 = cv2.filter2D(img,-1,k4)
img5 = cv2.filter2D(img,-1,k5)

plt.figure(figsize=(10,6))
plt.subplot(2,3,1),plt.imshow(img)
plt.subplot(2,3,2),plt.imshow(img1,cmap='gray')
plt.subplot(2,3,3),plt.imshow(img2,cmap='gray')
plt.subplot(2,3,4),plt.imshow(img3,cmap='gray')
plt.subplot(2,3,5),plt.imshow(img4,cmap='gray')
plt.subplot(2,3,6),plt.imshow(img5,cmap='gray')
```



Differentiate Edge Detection and Embossing

Feature	Edge Detection	Embossing
Purpose	To detect and highlight sharp intensity changes	To simulate a 3D raised or engraved look
Output	Binary or grayscale edges (lines)	Shaded relief effect, often "shadowy"
Technique	Uses gradient or second derivative (e.g., Sobel, Laplacian, Canny)	Uses directional kernels to highlight lighting contrast
Colour Handling	Often converts to grayscale	Often applied to grayscale, adds artificial shading
Visual Result	Thin lines where changes occur	Pseudo-3D look, makes surfaces look raised
Usage in ML / CV	Preprocessing for segmentation, OCR, object detection	Mostly for artistic or stylistic purposes
When to use?	Face detection, license plate recognition, object tracking, etc.	Stylistic filters, art applications, applications in which you want to give embossed effect.

Erode and dilate image (Morphological Operations)

Erosion and Dilation are the two basic morphological operations used in image processing. They are mainly applied on binary images (black & white) but can also work on grayscale images.

Erosion (Image Shrinking):

- Erosion strips the outermost layer of pixels in the structure of an image.
- Shrinks white regions, removes noise, makes objects thinner.
- Erosion means eating away the boundaries of the foreground (white) objects.
- The structuring element slides over the image.
- A pixel in the output is white (1) only if all pixels under the kernel are white.
- Otherwise, it is set to black.
- Erosion results into smaller and thinner foreground objects.
- The built-in function erode() is used to perform the same and its argument iterations specify the number of time operations/layer is applied to the resultant image.

Example:

Original binary image (1 = white, 0 = black):	After erosion with a 3×3 kernel:
0 0 0 0 0 0 1 1 1 0 0 1 1 1 0 0 1 1 1 0 0 0 0 0 0	0 0 0 0 0 0 0 1 0 0 0 1 1 1 0 0 0 1 0 0 0 0 0 0 0

We can find that after applying erosion, the object has become thinner.

Dilation (Image Expanding)

- Dilation means adding pixels to the boundaries of the foreground objects.
- A pixel in the output is white (1) if any pixel under the kernel is white.
- Otherwise, it is set to black.
- Dilation results into bigger and thicker foreground objects.
- The built-in functions dilate() is used to perform the same and its argument iterations specify the number of time operations/layer is applied to the resultant image.

Example:

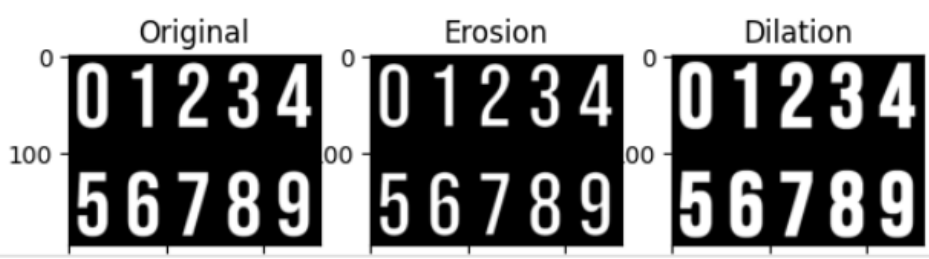
Original binary image (1 = white, 0 = black):	After erosion with a 3×3 kernel:
0 0 0 0 0	0 1 1 1 0

0 1 1 1 0 0 1 1 1 0 0 1 1 1 0 0 0 0 0 0	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0

We can find that after applying dilation, the object has become thicker.

```
#Erosion and Dilation
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image (binary or grayscale)
img = cv2.imread('0to9.png', 0)
img = cv2.bitwise_not(img) # To invert image

# Define a 3x3 kernel
kernel = np.ones((3,3), np.uint8)

# Apply erosion
erosion = cv2.erode(img, kernel, iterations=1)

# Apply dilation
dilation = cv2.dilate(img, kernel, iterations=1)

# cv2.imshow('Original', img)
# cv2.imshow('Erosion', erosion)
# cv2.imshow('Dilation', dilation)
plt.subplot(1,3,1), plt.imshow(img, cmap='gray'), plt.title("Original")
plt.subplot(1,3,2), plt.imshow(erosion, cmap='gray'), plt.title("Erosion")
plt.subplot(1,3,3), plt.imshow(dilation, cmap='gray'), plt.title("Dilation")
# cv2.waitKey(0)
# cv2.destroyAllWindows()
```

```
(<Axes: title={'center': 'Dilation'}>,
 <matplotlib.image.AxesImage at 0x2957bf24430>,
 Text(0.5, 1.0, 'Dilation'))
```



Differentiate Erosion and Dilation

Feature	Erosion	Dilation
Effect	Shrinks white regions	Expands white regions

Feature	Erosion	Dilation
Operation	Pixel = min in neighbourhood	Pixel = max in neighbourhood
Removes	Small white noise	Small black holes
Object Boundaries	Move inwards (thin objects)	Move outwards (thick objects)
Applications	Noise removal, separation	Filling gaps, connecting objects

Applications

- Removing noise from scanned documents.
- Filling gaps or cracks in objects.
- Separating connected objects (using erosion).
- Joining broken parts of characters (using dilation).

Contrast effect on gray scale image

What is Contrast in Gray scale?

- Contrast means the difference between the darkest and brightest pixel intensities in an image.
- In grayscale images, pixel values range from 0 (black) to 255 (white).
- If an image uses a wide range of values than the image is high contrast.
- If an image uses a narrow range where all pixels are mid-gray than the image is low contrast.
- To apply a contrast effect on the image, we have one way known as Histogram Equalization.
- According to this way, the pixel values are adjusted in the range of 0 to 255.
- Histogram Equalization (HE) spreads the pixel intensities more uniformly across 0–255, so dark regions get darker, bright regions get brighter details become easier to see.
- This filter is applicable to grayscale images as it works with intensity values in the image.

Low contrast image and High contrast image

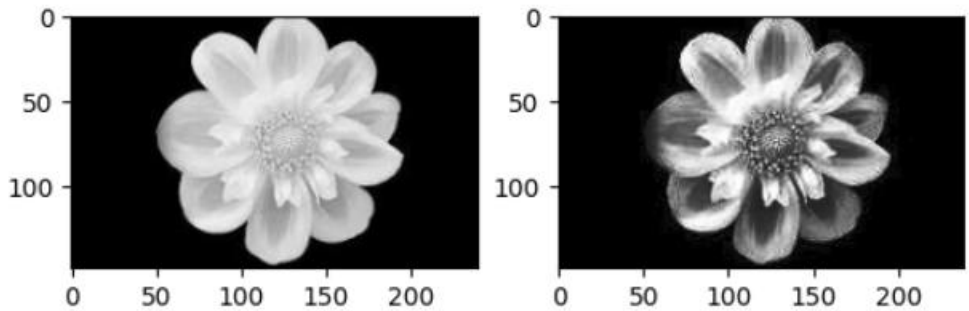
Low Contrast image	High Contrast image
Pixel values lie in a narrow range (say 100–150).	Pixel values span a wide range (say 20–230).
Image looks “dull” or “foggy”.	Image looks sharper.
Hard to distinguish details.	Clear separation between dark and bright areas.
110 115 120 118 122 125 119 121 123 All values are close, image looks flat.	20 50 90 150 200 230 100 180 220 Images look vivid.

Global Histogram Equalization (GHE)

- In grayscale, we stretch pixel intensities (0–255) so the histogram becomes more spread out.
- By this it improves contrast by making darks darker and lights lighter.
- For using Global Histogram Equalization, we will use '*equalizeHist(source)*'.

```
#Contrast effect on gray scale image
import cv2
import matplotlib.pyplot as plt
import numpy as np

img = cv2.imread('g_flower.jpg',0)
img1 = cv2.equalizeHist(img)

plt.subplot(1,2,1),plt.imshow(img,cmap='gray')
plt.subplot(1,2,2),plt.imshow(img1,cmap='gray')
```

```
(<Axes: >, <matplotlib.image.AxesImage at 0x202acecfbe0>)
```



Contrast effect on color image

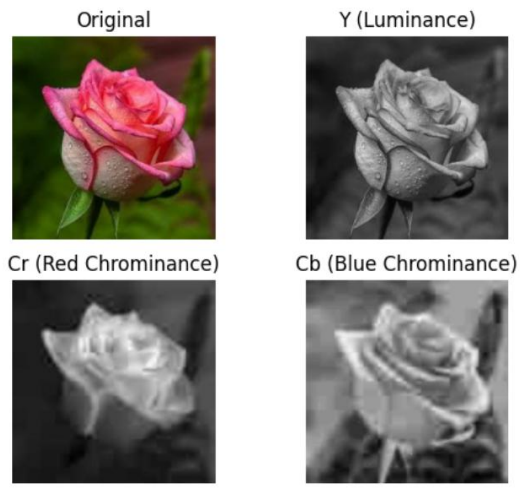
What is Contrast in colour image?

- In colour images, contrast refers to differences in brightness and colour between regions, making objects distinguishable.
- A low-contrast colour image looks flat, faded, or foggy.
- A high-contrast colour image looks vivid and well-separated.
- In OpenCV, colour image has 3 channels (RGB or BGR).
- Simply equalizing each channel separately, shifts colours unnaturally
- But the better approach is to work on a luminance channel (brightness) while keeping color the same.

YCrCb

- Y (Luminance Channel), Shows brightness and structural details.
- Chrominance red represents how much red is in each pixel, separate from the brightness.
- Chrominance blue represents how much blue is in each pixel, separate from the brightness.
- Y controls detail/brightness.
- Cr + Cb provide colour tint.
- When combined, we will get the full natural-looking colour image.

```python
#Showing effect of Y (Luminance), Cr (Red Chrominance) and Cb (Blue Chrominance)
import cv2
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("rose.jpg")

# Convert to YCrCb
ycrcb = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
y, cr, cb = cv2.split(ycrcb)

# Plot
plt.figure(figsize=(6,5))
plt.subplot(2,2,1); plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)); plt.title("Original"),plt.axis('off')
plt.subplot(2,2,2); plt.imshow(y, cmap='gray'); plt.title("Y (Luminance)"),plt.axis('off')
plt.subplot(2,2,3); plt.imshow(cr, cmap='gray'); plt.title("Cr (Red Chrominance)"),plt.axis('off')
plt.subplot(2,2,4); plt.imshow(cb, cmap='gray'); plt.title("Cb (Blue Chrominance)"),plt.axis('off')
```



Global Histogram Equalization (GHE)

- Global Histogram Equalization is a contrast enhancement technique where the entire image is considered at once to redistribute pixel intensities more evenly across the available range.
- "Global" means one mapping function is applied to the whole image.
- "Equalization" means stretches pixel values so that the histogram is more uniform.
- As a result, details become more visible, and contrast increases.
- If you apply GHE separately to each colour channel (R, G, B), the result looks unnatural.
- Colours shift because each channel is stretched independently.
- For color images, we don't equalize RGB directly.
- Instead, we convert to a color space that separates luminance (brightness) from chrominance (color), it means we will use YCrCb.
    - Steps to follow:
        - Convert BGR to YCrCb.
        - Apply GHE only on Y (luminance).
        - Merge back 'Cr' and 'Cb' unchanged.
        - Convert back to BGR for display.

```python
#Contrast effect on color image
#Using YCrCb on Global Histogram Equalization
import cv2
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("rose.jpg")

# Convert to YCrCb
ycrcb = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
y, cr, cb = cv2.split(ycrcb)

# Global Histogram Equalization
y_eq = cv2.equalizeHist(y)
ycrcb_eq = cv2.merge([y_eq, cr, cb])
global_eq = cv2.cvtColor(ycrcb_eq, cv2.COLOR_YCrCb2BGR)

# Plot images
plt.figure(figsize=(6,5))
plt.subplot(1,2,1),plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)),plt.title("Original Image"),plt.axis('off')
plt.subplot(1,2,2),plt.imshow(cv2.cvtColor(global_eq, cv2.COLOR_BGR2RGB)),plt.title("Global Equalization"),plt.axis('off')
```

```
(<Axes: title={'center': 'Global Equalization'}>,
 <matplotlib.image.AxesImage at 0x1b2d8049570>,
 Text(0.5, 1.0, 'Global Equalization'),
 (np.float64(-0.5), np.float64(224.5), np.float64(224.5), np.float64(-0.5)))
```

CLAHE – Contrast Limited Adaptive Histogram Equalization

- It is an improved version of global histogram equalization.
- Instead of applying one global mapping to the whole image, CLAHE:
  - Divides the image into small tiles (regions).
  - Applies histogram equalization locally within each tile.
  - Applies a clip limit to prevent over enhancement (too much noise/contrast).
- It Enhances local contrast instead of global.
- Keeps natural appearance while revealing hidden details.

Working style of CLAHE

- Divide the image into small blocks (tiles), e.g., 8×8.
- Limit contrast (clip limit):
- If some Gray levels dominate (too many pixels), CLAHE redistributes them to avoid noise.
- Interpolate between neighbouring tiles so the image looks smooth.

```python
#Contrast effect on color image
#Using YCrCb on CLAHE Equalization
import cv2
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("rose.jpg")

# Convert to YCrCb
ycrcb = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
y, cr, cb = cv2.split(ycrcb)

# CLAHE (adaptive)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
y_clahe = clahe.apply(y)
ycrcb_clahe = cv2.merge([y_clahe, cr, cb])
clahe_eq = cv2.cvtColor(ycrcb_clahe, cv2.COLOR_YCrCb2BGR)

# Plot images
plt.figure(figsize=(6,5))

# Plot images
plt.figure(figsize=(6,5))
plt.subplot(1,2,1),plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)),plt.title("Original Image"),plt.axis('off')
plt.subplot(1,2,2),plt.imshow(cv2.cvtColor(clahe_eq, cv2.COLOR_BGR2RGB)),plt.title("CLAHE Equalization"),plt.axis('off')
```

Explanation of parameters used

clipLimit: limits contrast amplification.

- Low values = less enhancement (safer).
- High values = stronger enhancement (risk of noise).
- Common: 2.0 to 4.0.

tileGridSize: number of tiles (e.g., (8,8)).

- Smaller tiles = more local contrast enhancement.
- Larger tiles = closer to global HE.

Difference between Global Equalization and CLAHE

Method	Effect
Global HE	Improves overall contrast but may over-enhance some areas and add noise.
CLAHE	Improves local contrast adaptively, producing more natural and balanced results.

Applications of image contrast

- Photography (improve dull photos)
- Satellite/remote sensing
- Medical images (MRI/CT scans in colour maps)
- Surveillance (low-light video enhancement)

Applying advanced image filters

1). Bilateral (for advanced smoothing effect)

```
# Applying advanced image filters
# 1). Bilateral (for advanced smoothing effect)
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("rose.jpg")
bilateral = cv2.bilateralFilter(img, d=9, sigmaColor=75, sigmaSpace=75)

plt.subplot(1,2,1), plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)), plt.title("Original"), plt.axis('off')
plt.subplot(1,2,2),plt.imshow(cv2.cvtColor(bilateral, cv2.COLOR_BGR2RGB)), plt.title("Bilateral Filtered"), plt.axis('off')
```

```
(<Axes: title={'center': 'Bilateral Filtered'}>,
 <matplotlib.image.AxesImage at 0x15ad04fd240>,
 Text(0.5, 1.0, 'Bilateral Filtered'),
 (np.float64(-0.5), np.float64(224.5), np.float64(224.5), np.float64(-0.5)))
```

Syntax:

cv2.bilateralFilter(src, d, sigmaColor, sigmaSpace)

Explanation of the syntax:
src: input image
d: diameter of pixel neighbourhood
sigmaColor: filter sigma in colour space
Larger value, stronger smoothing, can blur edges.
Smaller value, less smoothing, can preserve edges.
sigmaSpace: filter sigma in coordinate space
Larger value, more distant pixels considered.
Smaller value, only nearby pixels influence.

2) Unsharp (for advanced sharpening)

- Despite the name "unsharp", it is a sharpening technique

Syntax:

Sharpened Image=Original+k(Original−Blurred)

Putting values in to the syntax

Sharpened Image = cv2.addWeighted(img, 1.5, gauss, -0.5, 0)

Explanation of the syntax:
Takes a weighted sum of two images:

- 1.5 × Original image (increase original details).
- -0.5 × Blurred image (subtract the blurred part).

```python
# 2). Unsharp (for advanced sharpening)
import cv2

# Load image
img = cv2.imread("rose.jpg")

# Blur the image
gauss = cv2.GaussianBlur(img, (9,9), 10)

# Unsharp masking
unsharp = cv2.addWeighted(img, 1.5, gauss, -0.5, 0)

plt.subplot(1,2,1), plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)), plt.title("Original"), plt.axis('off')
plt.subplot(1,2,2),plt.imshow(cv2.cvtColor(unsharp, cv2.COLOR_BGR2RGB)), plt.title("Unsharp Masked"), plt.axis('off')
```

```
(<Axes: title={'center': 'Unsharp Masked'}>,
 <matplotlib.image.AxesImage at 0x15ad04fe350>,
 Text(0.5, 1.0, 'Unsharp Masked'),
 (np.float64(-0.5), np.float64(224.5), np.float64(224.5), np.float64(-0.5)))
```

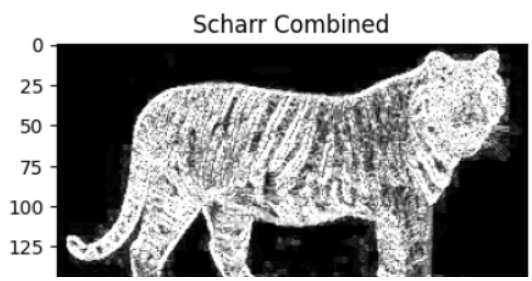
3) Scharr for better edge detection

- Scharr is a gradient-based edge detection filter like Sobel.
- Like Sobel, it finds edges by measuring intensity change in the image.
- Difference: Scharr provides better accuracy in detecting diagonal/rotated edges, because it uses a more precise kernel.

Syntax:

cv2.Scharr(src, ddepth, dx, dy[, scale[, delta[, borderType]]])

```python
import cv2
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("tiger.jpg", 0)  # grayscale

# Apply Scharr filters
scharrx = cv2.Scharr(img, cv2.CV_64F, 1, 0)  # vertical edges
scharry = cv2.Scharr(img, cv2.CV_64F, 0, 1)  # horizontal edges

# Convert to uint8 for display
scharrx = cv2.convertScaleAbs(scharrx)
scharry = cv2.convertScaleAbs(scharry)

# Combine both directions
scharr_combined = cv2.addWeighted(scharrx, 0.10, scharry, 0.2, 0)

# Show results
plt.figure(figsize=(15,15))
# plt.subplot(1,3,1), plt.imshow(scharrx, cmap='gray'), plt.title("Scharr X")
# plt.subplot(1,3,2), plt.imshow(scharry, cmap='gray'), plt.title("Scharr Y")
plt.subplot(1,3,3), plt.imshow(scharr_combined, cmap='gray'), plt.title("Scharr Combined")
plt.show()
```



Object Detection
Face detection

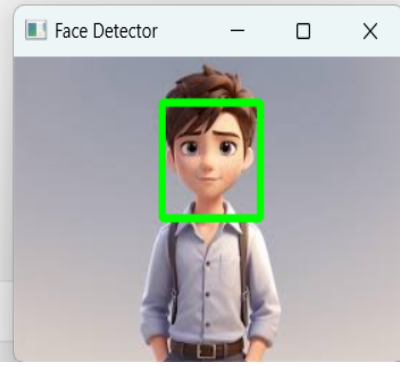
- Object detection, within computer vision, involves identifying objects within images or videos.
- These algorithms commonly rely on machine learning or deep learning methods to generate valuable outcomes.

What is Haar Cascade?

- It is an Object Detection Algorithm used to identify faces in an image or a real time video.
- Haar cascade is an algorithm that can detect objects in images, irrespective of their scale in image and location.
- This algorithm is not so complex and can run in real-time.
- We can train a haar-cascade detector to detect various objects like cars, bikes, buildings, fruits, etc.
- Haar Cascade Detection is one of the oldest, yet powerful face detection algorithms invented.
- Haar Features were not only used to detect faces, but also for eyes, lips, license number plates etc.
- The models are stored on GitHub, and we can access them with OpenCV methods.

```python
import cv2
# Load pre-trained face detector (Haar Cascade)
face_cascade=cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
# Read image
img=cv2.imread('Person.png')
gray_image=(cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)) #Haar works on 8-bit single-channel images, it is required to convert to gray.
# Detect faces
faces=face_cascade.detectMultiScale(gray_image,scaleFactor=1.1,minNeighbors=4)
print(faces)
# Draw rectangles around faces
for (x,y,w,h) in faces:
    cv2.rectangle(img,(x,y),(x+w,y+h),(0,255,0),3)
cv2.imshow('Face Detector',img)
cv2.waitKey()
cv2.destroyAllWindows()
```

```
[[104  27  69  69]]
```



About functions and parameters used in the code:

CascadeClassifier('haarcascade_frontalface_default.xml')
The above line creates a Haar cascade face detector from an XML file. This XML file contains a pre-trained model to detect face.

facecascade.detectMultiScale(gray_image,scaleFactor=1.1,minNeighbors=4)
detectMultiScale, At each window position & size, it checks: "Does this look like a face?"

scaleFactor=1.1 means builds an image pyramid; at each step the image is shrunk by 10%. Smaller values mean finer scale search (more accurate, slower). Larger values mean faster, might miss faces.

minNeighbors=4 means how many nearby detections are needed to accept a face. Higher mean fewer false positives but may miss faint faces.

for (x,y,w,h) in faces:
Each detected face is returned as a rectangle:

- x = top-left corner (horizontal position)
- y = top-left corner (vertical position)
- w = width of the face rectangle
- h = height of the face rectangle

cv2.rectangle(img,(x,y),(x+w,y+h),(0,255,0),3)
img: the image on which to draw.
(x, y): top-left corner of the rectangle.
(x+w, y+h): bottom-right corner.

- Since we know only top-left and width/height, we add w to x and h to y.

(0,255,0): rectangle color (BGR format = Green here).
3: thickness of the rectangle border in pixels.

```python
#Detecting faces from Camera
import cv2
# Load Haar Cascade for face detection
face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")
# Open webcam (0 = default camera, 1 = external camera)
cap = cv2.VideoCapture(0)

while True:
    # Capture frame-by-frame
    ret, frame = cap.read()
    if not ret:
        break  # if camera not available

    # Convert to grayscale (Haar works better on grayscale)
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Detect faces
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5)

    # Draw rectangles around faces
    for (x, y, w, h) in faces:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

    # Display the resulting frame
    cv2.imshow('Webcam Face Detection', frame)

   #Close the capture by pressing the 'esc'.
    if cv2.waitKey(1) == 27:  # 27 is ASCII for ESC
        break

# Release the capture and close windows
cap.release()
cv2.destroyAllWindows()
```